

O Codificador Limpo:

Um Código de Conduta para Programadores Profissionais



Capítulo 10: Estimativa

**Este material foi desenvolvido pelo grupo
PET/ADS do IFSP São Carlos**

O Medo das Estimativas

A **estimativa** é uma das atividades mais **simples** e, ao mesmo tempo, mais **assustadoras** para o profissional de software.

Grande parte do **valor de negócio** depende dela, mas a falta de precisão gera angústia, falhas e desconfiança mútua.

A estimativa é a fonte de quase todo o atrito na relação entre quem gerencia o produto e quem escreve o código.

Lições do Passado

Em 1978, Uncle Bob subestimou drasticamente um projeto de hardware, prometendo um mês para algo que levou três.

Sob o efeito do álcool em uma festa da empresa, ele confessou ao chefe seu pavor e insegurança sobre o atraso.

A resposta do gestor foi clara: ele via o esforço e o progresso, e embora estivesse demorando, o projeto era necessário.

Divergências de Percepção

A área de **negócios** enxerga **datas** como **compromissos** , enquanto **desenvolvedores** as tratam como meros **palpites** .

Essa divergência de interpretação é o fator principal que costuma **arruinar** a **credibilidade** de equipes inteiras.

A **desconfiança** mútua **nasce** quando **não há clareza** sobre o que cada lado entende por **prazo final**.

O atrito entre gerentes e técnicos é alimentado por essa **falta de alinhamento** sobre as incertezas.

O Compromisso

Um compromisso é **algo que deve ser alcançado** . Se você **prometeu** uma data, **deve entregar** naquela data.

Compromissos devem ser **assumidos** apenas quando houver **confiança real de entrega** .

Profissionais não **assumem compromissos** a menos que tenham **certeza absoluta** de que podem cumprir.

A Estimativa

Uma estimativa é um **palpite** . Não há promessa implícita ou honra em jogo caso o prazo não seja exato.

Estimamos justamente porque não sabemos quanto tempo algo levará. O **erro** é tratar a **estimativa** como um **número fixo** .

Uma **estimativa** real é uma **distribuição de probabilidade** , mostrando **cenários** do mais **provável** ao mais **pessimista** .

Estimativa ≠ Compromisso

Uma **estimativa** representa uma **probabilidade** baseada nas informações disponíveis naquele momento.

Já um **compromisso** é uma **promessa** assumida conscientemente pelo profissional.

Confundir esses dois conceitos cria expectativas irreais e compromete a confiança da equipe.

O profissional deve **comunicar** claramente **quando** está **estimando** e **quando** está se **comprometendo** .

Aceitar prazos **sem** essa **clareza técnica** compromete a honestidade profissional e a **qualidade do software** .

Implicações do "Tentar"

Dizer que vai "tentar" cumprir um prazo curto soa inocente, mas é um termo carregado de **perigo**.

Ao aceitar "**tentar**", o programador está secretamente **se comprometendo** com o **sucesso** daquela data impossível.

Se você **não** pretende **trabalhar** fins de semana ou mudar seus planos pessoais, você **não** está realmente "**tentando**".

O **termo** pode **gerar expectativas incompatíveis** com a realidade do projeto.

Negociação Profissional

Profissionais **não aceitam prazos impossíveis** apenas para evitar conflitos com a gestão.

Prazo, **escopo** e **qualidade** precisam ser discutidos de forma **transparente** e **racional**.

Quando uma **data não é viável**, o correto é **renegociar expectativas** e **apresentar alternativas técnicas**.

A **honestidade** na **negociação** preserva a **credibilidade** da equipe e **reduz riscos** no projeto.

Técnica PERT

Criada em 1957 para o projeto Polaris, a técnica PERT ajuda a converter palpites em distribuições matemáticas.

Para cada tarefa, o profissional deve fornecer três números distintos:

- **Otimista** : Tudo dá certo e a tarefa flui perfeitamente (chance $< 1\%$).
- **Nominal** : O tempo com a maior probabilidade de ocorrer na prática.
- **Pessimista** : Tudo dá errado, incluindo desastres e imprevistos técnicos.

Calculando a Probabilidade: média ponderada

Utilizamos as variáveis Otimista (**O**), Nominal (**N**) e Pessimista (**P**), pois elas cobrem desde o cenário ideal até as maiores dificuldades técnicas.

A média ponderada (μ) gera uma **expectativa de entrega** condizente com a realidade:

$$\mu = (O + 4N + P)/6$$

Calculando a Probabilidade: desvio padrão

O desvio padrão (σ) mede a **incerteza e o risco** matemático da tarefa:

$$\sigma = (P - O)/6$$

Um **desvio alto** sinaliza **falta de clareza** ou **alta complexidade técnica**. Dessa forma, a estimativa deixa de ser um palpite e vira uma **análise de riscos**.

Estimando o Projeto

Para **estimar** um **projeto** completo, a duração final esperada é a **soma das médias individuais** (μ) de cada tarefa mapeada.

Este cálculo garante que os prazos não sejam baseados em desejos, mas sim em uma base matemática realista e sólida.

A **incerteza** acumulada do projeto é a **raiz quadrada da soma dos quadrados dos desvios padrão** (σ) de todas as atividades.

Esse método **impede** que o **otimismo isolado** em certas tarefas **mascare os riscos** reais de atraso no cronograma geral.

Estimando o Projeto - Exemplo

História do Usuário

- Como cliente, desejo recuperar minha senha por e-mail para voltar a acessar minha conta caso eu a esqueça.

Estimativas	Estimativa PERT	Desvio Padrão (σ)
• Otimista (O) = 2 horas	$\mu = (O + 4N + P)/6$	$\sigma = (P - O)/6$
• Nominal (N) = 4 horas	$\mu = (2 + 4(4) + 8)/6$	$\sigma = (8 - 2)/6$
• Pessimista (P) = 8 horas	$\mu = 26/6 = 4.333$ $4.333 = 4 \text{ horas e } 20 \text{ min}$	$\sigma = 1 \text{ hora}$

Estimando o Projeto - Exemplo

Incerteza Acumulada

Exemplo		Incerteza
Atividade	Desvio (σ)	$\sqrt{A\sigma^2 + B\sigma^2 + C\sigma^2 + D\sigma^2}$
A	0,5 h	$\sqrt{0,5^2 + 1,0^2 + 0,6^2 + 0,8^2}$
B	0,8 h	
C	0,6 h	
R.Senha	1,0 h	$\sqrt{0,25 + 1,00 + 0,36 + 0,64}$
		$\sqrt{2,25} = 1,5 = 1\text{h } 30\text{m}$

Estimando em Equipe

O recurso de estimativa **mais importante** são as **pessoas** ao seu redor, pois elas **veem detalhes** que **você** pode **ignorar**.

Profissionais **evitam palpites isolados** e buscam validar suas suposições técnicas com o restante da equipe de desenvolvimento.

Técnicas de consenso, como o Wideband Delphi, impedem que um único desenvolvedor assuma o fardo de um prazo impossível.

O **time discute** a implementação e itera os **valores de esforço** até que todos os membros cheguem a um **acordo razoável**.

Métodos Práticos

Existem **diversas formas** de **alcançar** o **consenso** sem a interferência da opinião de terceiros:

- **Flying Fingers** : Membros mostram dedos de 0 a 5 simultaneamente para indicar o esforço.
- **Planning Poker** : Uso de cartas com valores (frequentemente Fibonacci) para definir o tamanho da tarefa.
- **Affinity Estimation** : Organização silenciosa de cartões por tamanho relativo em uma mesa ou parede.

Lei dos Grandes Números

Estimativas são inerentemente **falhas**, mas podem ser **gerenciadas dividindo o trabalho**.

Ao **quebrar** uma **tarefa grande** em **várias pequenas**, **erros** individuais de cálculo tendem a se **anular** na soma final.

Erros de estimativa tendem **mais** para a **subestimação** do que para a **superestimação**, por isso a compensação raramente é perfeita.

A **decomposição** não apenas aumenta a **precisão**, mas ajuda a **revelar surpresas técnicas ocultas** no início.

Fragmentar o escopo permite um **controle maior** sobre as variáveis desconhecidas do projeto.

Ética e Profissionalismo

O desenvolvedor profissional **não faz promessas** que **não pode cumprir** e não alimenta falsas esperanças.

Quando um **compromisso é necessário** , ele **fornece números sólidos e trabalha** para alcançá-los.

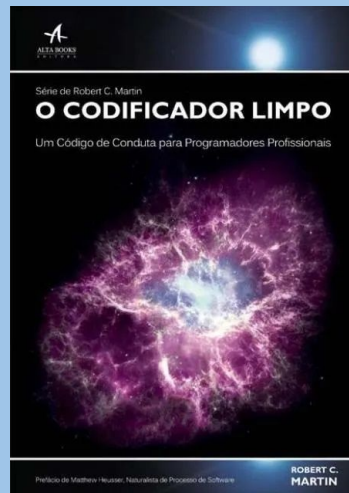
Em **todos os outros casos** , ele fornece **estimativas probabilísticas** que **descrevem a realidade técnica** do projeto.

Sobre o autor

Robert C. Martin é um engenheiro de software e consultor norte-americano com décadas de experiência na indústria. Conhecido como “Uncle Bob”, é um dos autores do Manifesto Ágil e escreveu livros influentes sobre boas práticas de programação, como O Codificador Limpo, baseados em sua experiência ensinando desenvolvedores a produzir código de qualidade.

Este conjunto de slides foi elaborado a partir da obra:

MARTIN, Robert C. **O codificador limpo:**
Um código de conduta para programadores
profissionais. 1. ed. Rio de Janeiro: Alta
Books, 2012.

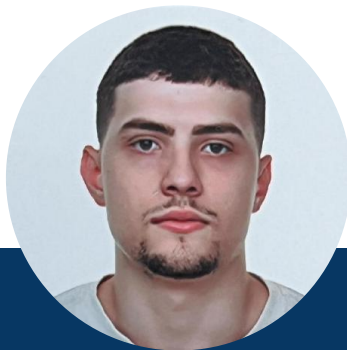


A equipe



Matheus Farias, Autor

Aluno do quarto período, é bolsista do PET/ADS desde outubro de 2024.



Cauã Habermann, Revisor

Aluno do terceiro período, é voluntário do PET/ADS desde novembro de 2025.



Lucas Oliveira, Tutor

Professor de Computação, é tutor do PET/ADS desde janeiro de 2023.